Building ETL Pipelines with Airflow

SoftUni Team

Technical Trainers

Software University

SoftUni

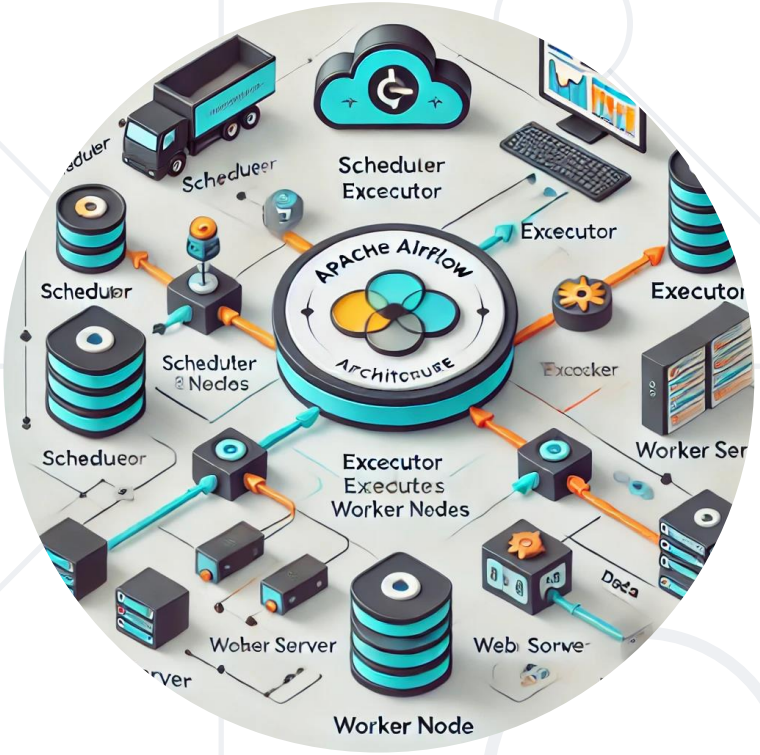
Software University

https://about.softuni.bg

Table of Contents

Table of Contents

Table of Contents



Apache Airflow

Introduction, Designing ETL Pipeline, Building ETL Pipeline and Advanced Features

What is Apache Airflow?

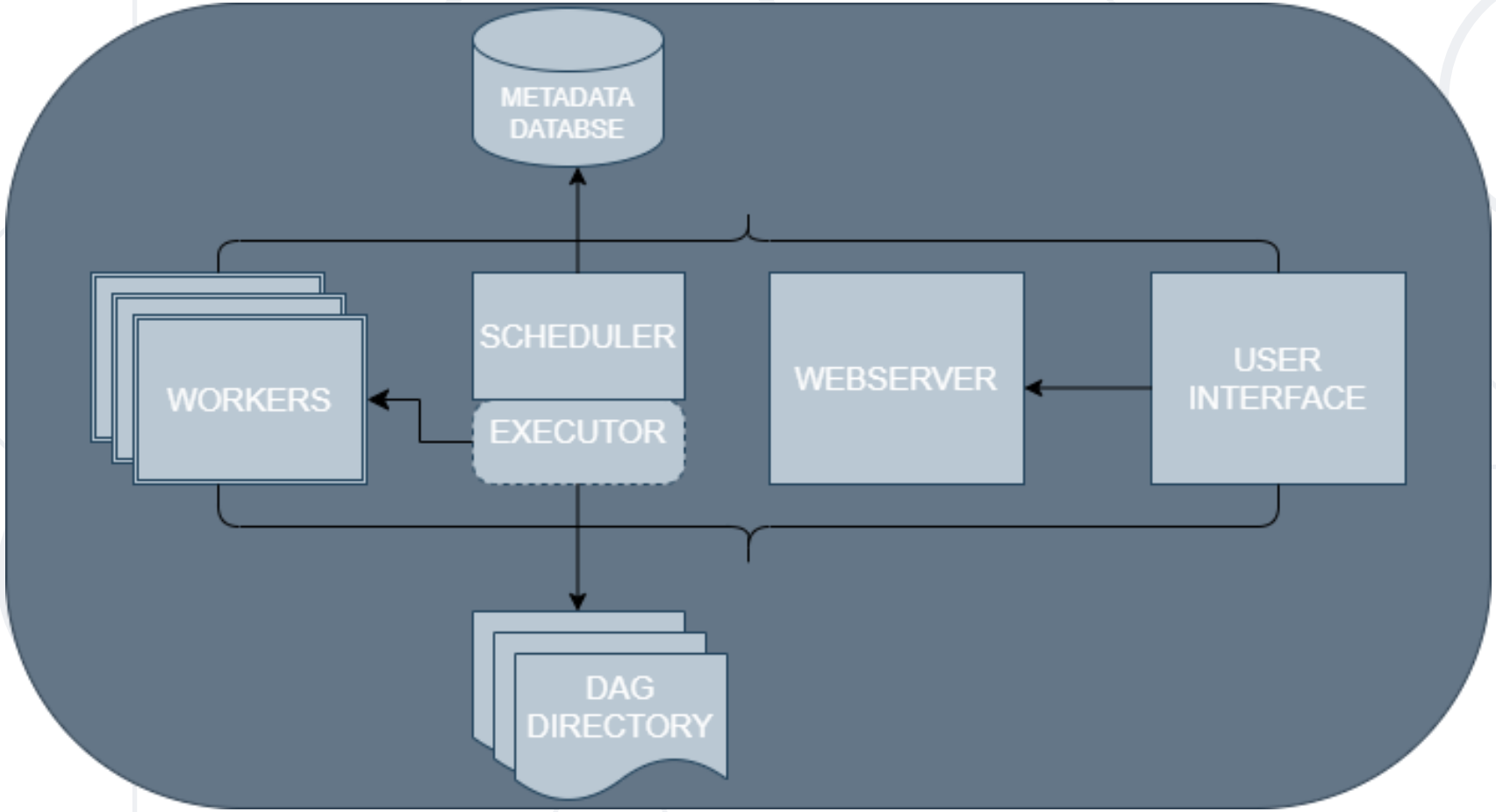
- Definition
  - A **Workflow Orchestration Platform** for **programmatically authoring**, **scheduling** and **monitoring workflows**
  - Designed for managing **ETL Pipelines**, **machine learning workflows** and more
  - **Directed Acyclic Graph** (DAGs) to define **workflows**
  - Components – **DAGs**, **Tasks**, **Operators**, **Scheduler** and **Webserver**

Key Components

- **Directed Acyclic Graphs** (DAGs) – Define workflows
- **Tasks** – Units of work within a DAG
- **Operators** – Predefined template for tasks (e.g., PythonOperators, BashOperators)
- **Scheduler** – Executes tasks based on dependencies and shedules
- **Webserver** – Provides a user interface to monitor and trigger DAGs

Schema

- Apache Airflow Architecture



What is TaskFlowAPI?

- **Simplifies ETL Pipeline** creation by allowing tasks to be defined as **Python functions**
- **Replaces** the need for defining tasks via **explicit operators** (e.g., PythonOperator)
- Uses the **@task** decorator to **define** tasks and **@dag** decorator to **define** the workflow

Why to use TaskFlowAPI?

Software University

- **Simplicity** and **Readability**
- **Automating Data Passing** with XComs
- **Better Debugging** and **Testing**
- **Python** – Native Integration
- **Dynamic Task Generation**
- **Reduced Boilerplace Code**

Comparison TaskFlowAPI to Traditional DAG

Software University

Feature	Traditional DAG	TaskFlowAPI
Easy of Use	Required explicit operat or definitions	Pythonic function-based syntax
Data Sharing	Manual XCom handling f or data sharing	Automatic XCom integrat ion between tasks
Readability	More verbose and fragm ented	Cleaner and more modul ar

Pipeline Design

- Extract – Fetch data from source

```
@task
def extract():
  sales_data = pd.read_csv("sales_data.csv")
  return sales_data
```

- Transform – Clean and aggregate the data

```
@task
def transform():
  total_sales = sum(item["sales"] for item in data)
  return {"total_sales": total_sales}
```

Pipeline Design

- Load – Save the data to source

```
@task
def load(aggregated_data):
     pd.to_csv(aggregated_data)
```

Task Dependencies

- Task Dependencies

```
sales_data = extract
aggregated_data = transform(sales_data)
load(aggregated_data)
```

Tradition DAG example

- Tradition DAG example

```
from airflow.operators.python import PythonOperator
def extract():
      print(“Extractiong data…”)
dag = DAG(dag_id=“traditional_dag”, start_date=datetime(2023, 1, 1))
extract_task = PythonOperator(task_id = “extract”,
python_callable=extract, dag=dag)
```

TaskFlowAPI

- TaskFlowAPI

```
from airflow.decorators import task, dag
@dag(schedule=None, start_date=datetime(2023, 1, 1))
      def taskflow_dag():
            @task
            def extract():
                  print("Extracting data….")
            extract()
taskflow_dag()
```

ETL Pipeline Structure

```
from airflow.decorators import dag, task
from datetime import datetime @dag(schedule_interval=None, start_date=datetime(2023, 1, 1))
def etl_pipeline():
     @task
     def extract():
           return {"sales": [100, 200, 300]}
     @task
     def transform(data):
          return sum(data["sales"])
    @task
    def load(total_sales):
         pd.to_csv(total_sales)
    data = extract()
    total_sales = transform(data)
    load(total_sales)
 etl_pipeline()
```

Dynamic Task Generation

- Dynamic Task Generation
  - Dynamically create tasks based on input

```
@task
def generate_tasks(data):
    for record in data:
        process_data(record)
```

Error Handling and Retries

- Error Handling and Retries
  - Add retry logic for failed tasks

```
@dag(schedule_interval=None, start_date=days_ago(1), catchup=False, default_args={
        'retries': 3,  'retry_delay': timedelta(minutes=2)})
def etl_with_retry():
    @task()
    def extract_data():
         raise Exception("Extraction failed!")
    @task()
    def load_data():
        print("Loading data...")
extract = extract_data()
load = load_data()
extract >> load
etl_with_retry()
```

Integrating with External Systems

- **Databases** – Use psycopg2 or SQLAlchemi for DB
- **APIs** – Use the requests library for fetching data
- **Cloud Storage** – S3, Azure Blob, GCS

Definition and Key Characteristics

- Definition
  - Operators are **pre-defined classes** in Airflow that **encapsulate individual units** of work (or tasks) to be executed in a **Directed Acyclic Graph** (DAG). They serve as templates that define what type of work is performed
- Key Characteristics
  - **Abstraction:** Encapsulate complex execution details (e.g., running a script, executing a query) behind a simple interface
  - **Configurability:** Operators are configured with parameters, such as command strings, Python callables, or SQL queries, making them flexible for various tasks
  - **Integration:** Many operators support integration with external systems (databases, APIs, etc.) and include built-in error-handling, logging, and retry features

Common Airflow Operators and Their Use Cases

- **PythonOperator**
  - Use Case: Execute custom Python functions or scripts
- **BashOperator**
  - Use Case: Execute shell commands
- **SQLOperator** / **BigQueryOperator** / **PostgresOperator**
  - Use Case: Interact with databases by executing SQL commands
- **EmailOperator**
  - Use Case: Send email notifications
- **Sensor Operators**
  - Use Case: Wait for external events or conditions

Airflow Dag with Operators

Software University

```
dag = DAG(
    dag_id='example_operator_dag',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False)
extract_task = PythonOperator(
    task_id='extract_task',
    python_callable=extract_data,
    dag=dag)
transform_task = PythonOperator(
    task_id='transform_task',
    python_callable=transform_data,
    provide_context=True,  # Required for accessing XComs via kwargs
    dag=dag)
load_task = BashOperator(
    task_id='load_task',
    bash_command='echo "Loading data: {{ ti.xcom_pull(task_ids=\'transform_task\')
}}"',
    dag=dag)
extract_task >> transform_task >> load_task
```

Traditional Operators vs TaskFlowAPI

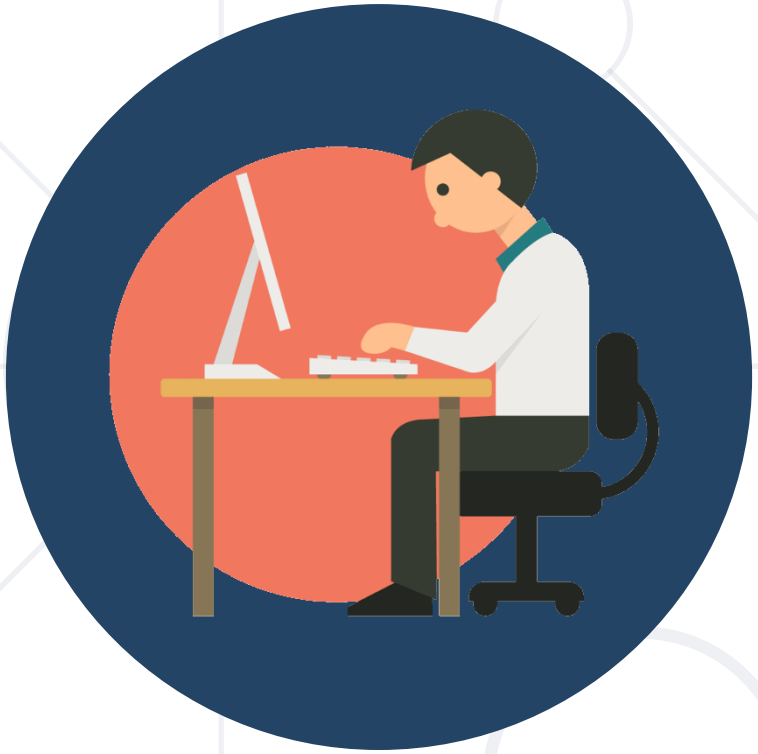
Software University

Aspect	Traditional Operators	TaskFlow API
Task Definition	Instantiate operator objects (e.g., PythonOperator, BashOperator)	Use @task decorator for defining tasks as simple Python functions
Boilerplate Code	Higher boilerplate (explicit instantiation and parameter setting)	Minimal boilerplate; logic resides in function bodies
Data Passing	Requires explicit use of XComs to share data between tasks	Automatically passes function outputs to downstream tasks
Dependency Management	Set via operators (>>, <<) separate from the task logic	Defined naturally through Python function calls, enhancing clarity
Dynamic Task Generation	More complex to generate dynamically; often requires loops with custom operators	Easily supports dynamic generation using native Python constructs
Readability & Maintenance	More verbose and less intuitive when workflow logic is embedded in operators	Clear and concise—closely resembles regular Python code, making it easier to read, test, and debug

Building and Running an ETL Pipeline - Practice

Software University

- Setup Airflow Environment with Astro
- Define the ETL Pipeline
- Run the Pipeline
- Explore Logs and Xcom



Practice

Exercise: Building ETL Pipelines with Airflow

Summary

- Understand how Airflow and TaskFlowAPI simplify ETL pipeline creation
- Be able to design, implement and execute a pipeline using the TaskFlowAPI
- Utilize advanced features like dynamic task generation, error handling and integrations for robust workflows